EmployeeHub – Architecture & Interview Guide

ASP.NET Core 10 Web API • Clean Architecture • SQL Server • Redis • JWT • Serilog • API Versioning • Testing

Purpose: A concise document to review before explaining the EmployeeHub project in an interview or technical discussion.

1. Project Overview

EmployeeHub is a layered employee-management application built with ASP.NET Core 10. The backend follows Clean Architecture principles so that business/application logic is separated from infrastructure details such as SQL Server, Entity Framework Core, Redis, and other external technologies.

The current backend is complete enough to demonstrate authentication, authorization, CRUD operations, validation, caching, logging, API versioning, unit testing, and integration testing. The planned next phase is a separate Angular 20+ frontend.

2. Solution Structure

```
EmployeeHub
■■■ EmployeeHub.Api
■■■ EmployeeHub.Application
■■■ EmployeeHub.Domain
■■■ EmployeeHub.Infrastructure
■■■ EmployeeHub.Api.Tests
■■■ EmployeeHub.PasswordGenerator
```

Layer responsibilities

Project	Responsibility
Domain	Core entities and domain model. No dependency on API or Infrastructure.
Application	Use cases/application logic, DTOs, service interfaces/implementations, repository/cache abstractions.
Infrastructure	EF Core, SQL Server, repositories, Redis implementation, external technology details.
API	Controllers, middleware, authentication configuration, DI/composition root, HTTP/OpenAPI configuration.
Api.Tests	Unit and integration tests.

3. Dependency Direction

```
API → Application → Domain
API → Infrastructure
Infrastructure → Application + Domain
```

Important rule: Domain does not depend on Application, Infrastructure, or API. Application does not depend on Infrastructure or API.

The API is the composition root: it connects interfaces to their concrete implementations using dependency injection.

4. Domain Layer

Entities currently include:
• Employee

• User
• Role

The Domain layer contains the core data model. For example, Employee contains Id, FirstName, LastName, Email, Department, Salary, and CreatedDate. User has a Role relationship, and Role contains its users.

5. Application Layer

```
Key folders:
DTOs: EmployeeDto, CreateEmployeeDto, UpdateEmployeeDto, LoginRequest, LoginResponse
Repositories: IEmployeeRepository, IUserRepository
Services: IEmployeeService, EmployeeService, IAuthService, AuthService
Caching: ICacheService
```

Application contains the logic that coordinates the application's use cases. EmployeeService performs DTO/entity mapping, validation, repository operations, caching, and logging. AuthService performs login logic, password verification, claim creation, and JWT creation.

6. Why Repository Interface and Implementation Are in Different Layers

```
Application
  IEmployeeRepository
    ↑ implemented by
Infrastructure
  EmployeeRepository
    ↓
  EF Core / AppDbContext
    ↓
  SQL Server
```

The Application layer needs data access, but it should not know how the data is stored. IEmployeeRepository is therefore an abstraction in Application. EmployeeRepository is Infrastructure because it depends on EF Core, AppDbContext, and SQL Server.

Services are different: EmployeeService and IEmployeeService both belong to Application because the service implementation contains application/business logic rather than a database implementation.

7. Infrastructure Layer

```
Data/AppDbContext.cs
Repositories/EmployeeRepository.cs
Repositories/UserRepository.cs
Caching/RedisCacheService.cs
```

Infrastructure implements Application abstractions. It contains EF Core configuration, SQL Server access, repository implementations, and the Redis cache implementation.

8. Dependency Injection / Composition Root

```
IEmployeeRepository → EmployeeRepository
IUserRepository → UserRepository
ICacheService → RedisCacheService
IEmployeeService → EmployeeService
IAuthService → AuthService
```

All of these connections are registered in Program.cs. Scoped lifetime is used for repositories and services. AppDbContext is also scoped, so a request normally uses a consistent DbContext within its request scope.

9. End-to-End Employee Request Flow

```
HTTP Request
  ↓
EmployeesController (API)
  ↓
IEmployeeService (Application abstraction)
  ↓
EmployeeService (Application logic)
  ↓
IEmployeeRepository (Application abstraction)
  ↓
EmployeeRepository (Infrastructure)
  ↓
AppDbContext / EF Core
  ↓
SQL Server
```

10. Caching Flow

```
EmployeeService
  ↓
ICacheService
  ↓
RedisCacheService
  ↓
IDistributedCache
  ↓
Redis
```

GET operations can use the cache first. When employee data is created, updated, or deleted, the employee cache is invalidated so stale data is not retained.

11. Authentication and Authorization

```
Login request
  ↓
AuthController
  ↓
IAuthService
  ↓
AuthService
  ■■■ IUserRepository → UserRepository → SQL Server
  ■■■ IPasswordHasher → verifies password hash
  ■■■ creates JWT
    ↓
Client receives token
```

JWT claims include username, role, and user ID. The EmployeesController requires authentication. Create, Update, and Delete operations require the AdminOnly policy, which requires the Admin role.

Typical authorization behavior: unauthenticated request → 401; authenticated User attempting an Admin-only operation → 403; authenticated Admin performing the operation → success.

12. Validation

CreateEmployeeDto and UpdateEmployeeDto use DataAnnotations such as Required, StringLength, EmailAddress, and Range. Salary must be at least $30,000. Invalid model state is returned through the configured validation response.

13. Logging and Error Handling

```
Request
  ↓
RequestLoggingMiddleware
  ↓
Controller / Service / Repository
  ↓
ExceptionHandlingMiddleware
  ↓
Consistent HTTP error response
```

Serilog writes structured logs to the console and daily files. RequestId and Username are included in the logging context. Passwords, password hashes, and JWT tokens are not logged.

14. API Versioning

The API uses URL-based versioning. The current employee endpoints use:

```
GET /api/v1/employees
```

This allows future versions such as v2 to evolve without changing the existing v1 contract. Scalar displays the versioned routes.

15. SQL Server and EF Core

AppDbContext is in Infrastructure and uses EF Core SQL Server. Salary is explicitly configured as DECIMAL(18,2) to avoid EF Core precision warnings and to match the SQL Server column.

16. Redis

Redis runs in Docker and is exposed on localhost:6379. The application uses Microsoft.Extensions.Caching.StackExchangeRedis and wraps the distributed cache behind ICacheService so Application does not depend directly on Redis.

17. Testing

Current status: 12 tests passing, 0 failing.
• Unit tests using xUnit/Moq
• Integration tests using WebApplicationFactory
• Authentication in integration tests uses TestAuthHandler
• Tests verify authenticated access, 401 behavior, 403 behavior, and Admin creation returning 201

Important current behavior: the Admin integration test performs a real POST through the application and currently uses the development EmployeeHub database. Therefore repeated test runs can create Integration/Test employee rows. This is intentional for the current learning setup; a dedicated test database can be introduced later.

18. Scalar / OpenAPI

Scalar is used instead of Swagger UI. The current API exposes OpenAPI information and Scalar for interactive API testing. JWT Bearer is configured in the OpenAPI document so authenticated endpoints can be tested through Scalar.

There are currently three API-versioning/OpenAPI analyzer warnings (AV0029/AV0030). They have intentionally been left for later because the application and tests are working correctly.

19. Interview Explanation – 60 Second Version

“EmployeeHub is an ASP.NET Core 10 Web API built using Clean Architecture. The Domain layer contains the core entities. The Application layer contains DTOs, application services, and repository/cache abstractions. Infrastructure implements

those abstractions using EF Core and SQL Server for persistence and Redis for caching. The API layer contains controllers, middleware, authentication configuration, and the dependency-injection composition root. Authentication uses JWT and password hashing, with role-based authorization for Admin operations. I also implemented validation, structured Serilog logging, API versioning, Redis caching, unit tests, and integration tests. The current API is versioned as /api/v1, and the next phase is an Angular 20+ frontend that will consume the API."

20. Key Interview Questions and Short Answers

Why Clean Architecture? To separate business/application logic from infrastructure and make the system easier to test, maintain, and change.

Why is IEmployeeRepository in Application? Application defines what data access it needs without knowing how that access is implemented.

Why is EmployeeRepository in Infrastructure? It depends on EF Core, AppDbContext, and SQL Server.

Why are IEmployeeService and EmployeeService both in Application? The service represents application/use-case logic, so its implementation belongs in the Application layer.

Why use dependency injection? To supply implementations to classes through abstractions and reduce direct coupling.

Why Redis? To avoid repeatedly retrieving suitable data from the database when cached data can safely be reused.

Why JWT? To authenticate API requests using a signed token that carries claims such as username and role.

What is the composition root? Program.cs, where the application connects abstractions to concrete implementations and configures the application pipeline.

21. Planned Frontend

```
EmployeeHub Backend
    ↑ HTTP/JSON
    ■
Angular 20+ Frontend
  ■■■ Login
  ■■■ Employee list
  ■■■ Employee create/edit/delete
  ■■■ JWT handling
  ■■■ Role-aware UI
```

The Angular application will be a separate client project. It will consume the existing EmployeeHub API rather than becoming another layer inside the backend Clean Architecture.

Final checkpoint

Backend status: Working and tested. The next development session should begin with the Angular 20+ frontend. Continue one small step at a time and test each step before moving forward.